

AI-Powered Event Services Marketplace for Kazakhstan

Diploma Defense — May 2026

Bahaidahar · Astana IT University

Problem

Event planning in KZ is broken

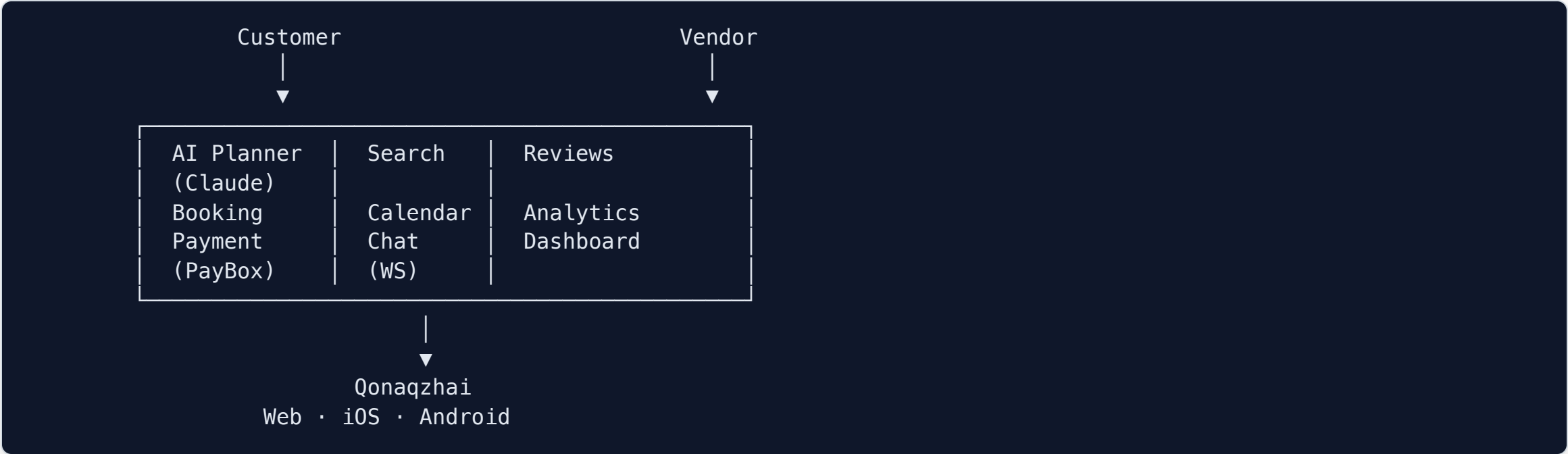
Today

- Discovery via Instagram DMs
- WhatsApp chains for quotes
- No verified reviews
- Deposit to private cards = trust risk
- Customer juggles 5–10 vendors
- No Kazakh-language platform
- Traditions ignored by global tools

Market

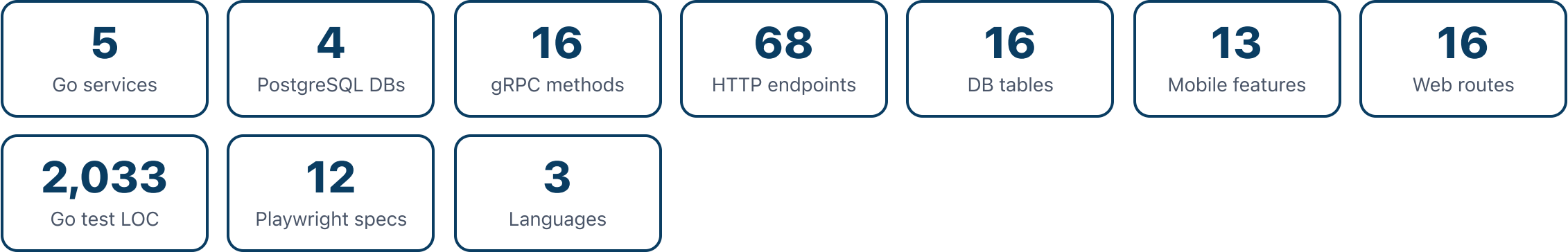
- KZ events sector ≈ **\$1.5B/year**
- Fragmented: Instagram, fl.kz, weddingsalon.kz
- No platform owns full journey

Solution at a Glance



One platform, three clients. KZ / RU / EN trilingual. AI-first.

Numbers



Stack

Backend

- Go 1.24
- gRPC
- Postgres

- 5 microservices: gateway / auth / core / payment / realtime
- Protobuf contracts inter-service
- HTTP/JSON at edge
- JWT, PayBox PSP

Web

- Next.js 16
- React 19
- FSD

- App Router, Cache Components, PPR
- Tailwind v4 · Playwright E2E

Mobile

- Flutter 3.24
- Riverpod
- MVVM

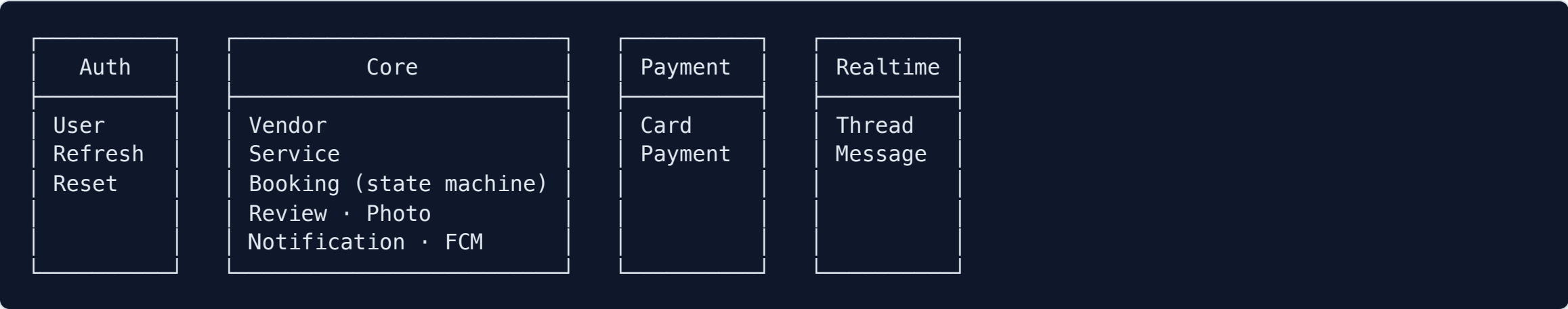
- Impeller engine · go_router
- Dio + auth interceptor
- Firebase Messaging

AI

- Claude Sonnet 4.6
- MCP

- Tool-use event planner
- Agentic dev workflow

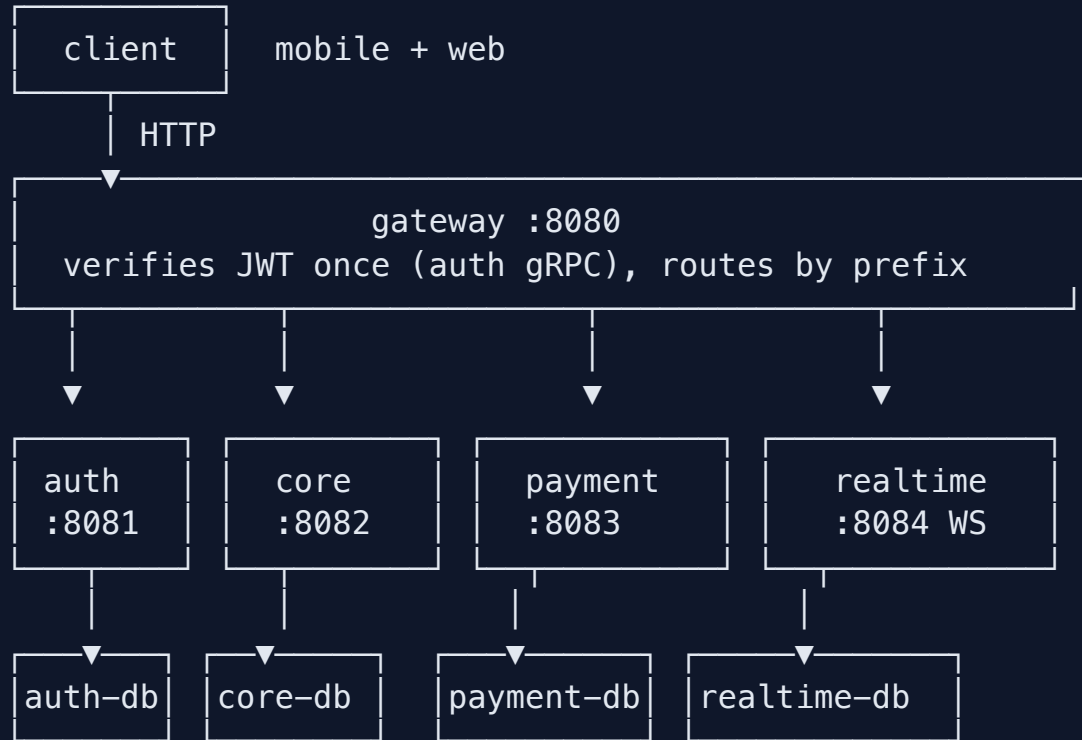
Domain Model



Booking lifecycle: pending → accepted → completed → paid

branches: declined, cancelled — explicit state machine in core/usecase/booking/

Backend — Five Go Microservices



One DB per service · gRPC inter-service · HTTP at edge

Service-to-Service Contracts (gRPC)

```
core —→ auth      GetUser · GetUsersBatch · VerifyToken
core —→ payment    Charge · Refund
core —→ realtime   EnsureThread · PublishEvent
payment —→ core     MarkBookingPaid      (saga callback)
realtime —→ auth    GetUsersBatch      (peer-name enrich)
gateway —→ auth     VerifyToken        (once per request)
```

Payment saga (synchronous)

```
core.booking.Pay → payment.Charge → core.MarkBookingPaid
```

Diploma scale — synchronous wins on simplicity. Async event bus (NATS / Kafka) is the production migration path.

Clean Architecture per Service

```
backend/services/core/internal/  
├── domain/      Pure types – Vendor, Booking, Review (no I/O)  
├── ports/       Interfaces – VendorRepo, PaymentClient, Notifier  
├── usecase/     Business logic – vendor/, booking/, review/, photo/  
└── adapter/  
    ├── http/    chi router, JSON marshalling, middleware  
    ├── grpc/    gRPC server impl (core.proto)  
    ├── grpcclient/ Outbound to auth, payment, realtime  
    ├── repo/    PostgreSQL (sqlx)  
    └── push/    FCM HTTP v1 sender
```

Layers depend inward only · `adapter` → `usecase` → `domain`

Dependency Inversion keeps usecase testable — table-driven Go tests stub the ports.

Web — Feature-Sliced Design

```
frontend/src/  
├── app/      Routes (App Router) – vendors, bookings, threads, admin  
├── features/ ai-chat · auth · booking-flow · messaging · reviews  
│           services · theme · vendor-search  
├── entities/ Domain types shared across features  
├── widgets/  Composed UI blocks  
└── shared/   UI kit · API client · i18n dictionary · hooks
```

Why FSD? Strict horizontal isolation between features. Dependency rule:

`shared → entities → features → widgets → app`

Next.js 16 features used — Cache Components with `"use cache"`, partial pre-rendering for vendor lists, React Server Components for vendor detail.

Mobile — Flutter, Riverpod, Clean Arch

```
mobile/lib/  
├── core/          network · router · theme · i18n · DI  
├── features/      13 modules – auth · vendor_catalog · booking  
│                 ai_chat · payment · cards · reviews · messaging  
│                 notifications · vendor_self · admin · onboarding  
│                 settings  
└── <feature>/  
    ├── data/      DTOs + datasources + repository impl  
    ├── domain/    Entities + abstract repo + use cases  
    └── presentation/ ViewModels (Riverpod) + screens
```

Riverpod compile-safe DI · no `BuildContext`

go_router declarative nav · deep-link ready

Dio + interceptor auto JWT refresh on 401

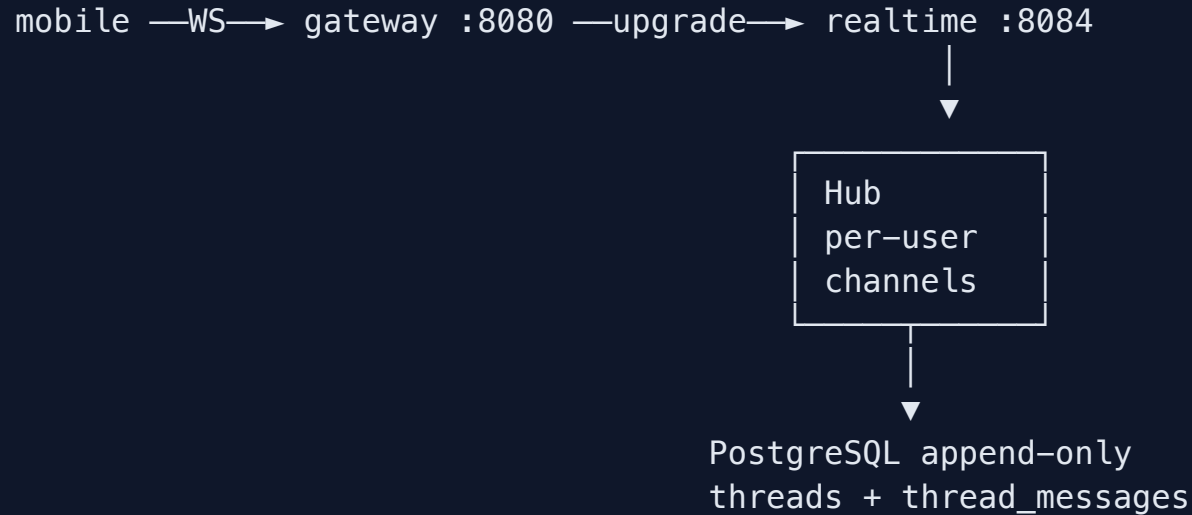
flutter_secure_storage Keychain / Keystore

Data Model

DB	Tables	Highlights
auth-db	users , refresh_tokens , password_reset_tokens	role enum, hashed tokens
core-db	vendors , services , photos , bookings , reviews , notifications , fcm_tokens	full-text GIN, rating aggregation, BYTEA photo blobs
payment-db	cards , payments	last4 + brand; PAN never persisted
realtime-db	threads , thread_messages	one thread per booking, append-only

No cross-service foreign keys. `core.bookings.customer_id` is a plain UUID — `auth-svc` owns the actual `users` row. Cross-refs resolved via batched gRPC `auth.GetUsersBatch`.

Real-time Chat (WebSocket)



- One thread per booking (`UNIQUE booking_id`)
- Hub fans out + persists immediately
- Auth at WS upgrade — gateway verifies JWT before upgrade
- Push fallback when peer offline — FCM deep-link `/threads/<id>`

Stack Justification — Comparative

Layer	Choice	Alternative	Why
Backend	Go 1.24	Node.js · Java	~2.6× CPU-bound · gRPC tooling
Inter-svc	gRPC + Protobuf	REST/JSON	5–10× throughput · 70–90% smaller payloads
Web	Next.js 16 + React 19	Remix · SvelteKit	Cache Components + PPR stable v16
Web arch	Feature-Sliced Design	Atomic · flat	Hard horizontal isolation
Mobile	Flutter 3.24 Impeller	React Native	46% vs 35% share · 60–120 fps
Mobile state	Riverpod	Bloc · Provider	Compile-safe DI · no BuildContext
DB	Postgres per service	Shared DB	Service autonomy · independent scaling
AI	Claude API + MCP	OpenAI raw	MCP is cross-vendor standard (Linux Foundation)
E2E	Playwright	Cypress	Parallel cross-browser · TS-native

Every choice backed by 2025–2026 industry data.

Competitive Landscape

Platform	Region	Model	AI	Mobile	KZ Traditions
The Bash	US	Commission	No	Yes	No
GigSalad	US	Freemium	Limited	Yes	No
Peerspace	US	Commission + escrow	Recs	Yes	No
The Knot	US	Listing + ads	Checklist	Yes	No
Wedding.ru	RU	Paid listings	No	Web-first	No
Eventie.kz	KZ	Paid listings	No	Web	Partial
weddingsalon.kz	KZ	Directory	No	Web	Partial
Qonaqzhai	KZ	Commission + escrow	Tool-use planner	Native Flutter	Yes

Qonaqzhai Differentiators

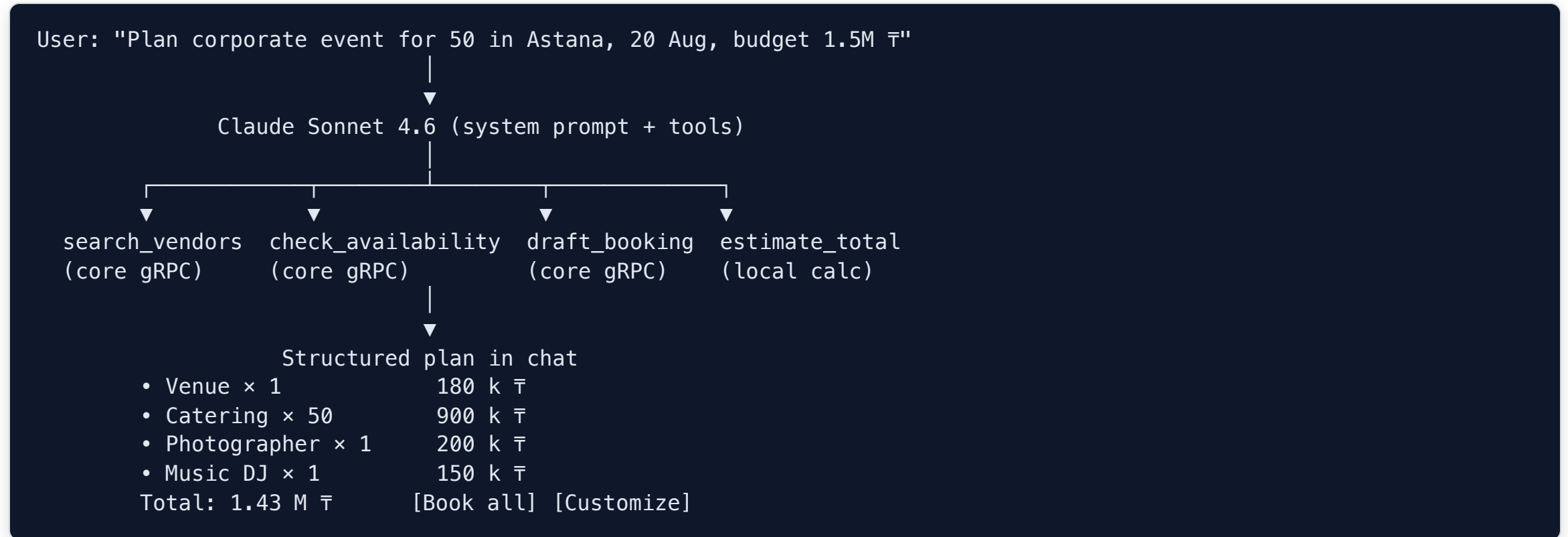
What competitors miss

- Kazakh first-class language
- Local PayBox + KZT
- AI planner with tool use
- Mobile-native, not web-first
- Tradition bundles (тұсауқесер · беташар · шашу)

Inspiration ported in

- Peerspace visual-first listings
- GigSalad freemium for vendors
- Knot budget tracker
- Eventbrite promoted slots
- Peerspace escrow with milestone release

Signature Feature — AI Planner with Tool Use



Anthropic Messages API · `tools` parameter · JSON schemas · SSE streaming from `core` service.

Security — Defense in Depth

Layer	Mechanism
Transport	HTTPS at gateway · Let's Encrypt
AuthN	JWT HS256 · 15 min access + 7 day refresh · hashed
AuthZ	RBAC middleware · per-route guards · ownership checks
Edge	Gateway calls <code>auth.VerifyToken</code> on every request
Service	Each service re-verifies JWT — no implicit trust
Rate limit	Per-IP 20–40 req / 10 s on every service
Password reset	One-time short-TTL tokens · SMTP delivery
PCI scope	PAN never persisted — PayBox PSP holds the scope
Photo uploads	MIME whitelist · 5 MB cap · content-type sniffed
Cross-DB joins	Forbidden — no shared schema, no leaked FKs

Testing & Quality

Component	Files	LOC	Approach
Auth service	1	386	Signup, login, refresh, password reset
Core service	4	1,007	Vendor CRUD, booking state machine, review aggregation
Payment service	2	436	Card validation, charge / refund, PSP mock
Realtime service	1	204	Thread ensure, message send, WS connection
Backend total	8	2,033	Table-driven Go tests · <code>-race</code> enabled
Frontend E2E	12 specs	~800	Playwright — auth, booking, chat, admin, vendor
Mobile	smoke	growing	Riverpod <code>ProviderContainer</code> mocks

CI gate — backend tests + Playwright E2E must pass before merge.

Agentic Engineering & MCP

How this diploma was built

MCP servers used in development

- **GitHub MCP** — branch / PR / issue automation
- **Filesystem MCP** — repo-wide refactors with safety checks
- **Postgres MCP** — schema introspection during migrations
- **Context7 MCP** — latest Next.js 16 + Riverpod docs
- **Exa / WebSearch** — competitor research, benchmark data

Why this matters in 2026

- Anthropic donated MCP to the **Linux Foundation Agentic AI Foundation** (2025)
- **10K+ MCP servers** live · **97M+ SDK installs**
- Stack Overflow 2025: **84%** of developers use AI coding tools
- IDC: **\$36.2B** AI dev-tool market by end-2026

Live Demo Flow (8 min)

1. **Localization** (30s) — Open mobile, switch KZ → RU → EN
2. **AI Planner** (2 min) — *"Plan corporate event for 50 in Astana, 20 Aug, 1.5M ₸"* → Claude returns 4-vendor plan with reasoning
3. **Booking + Calendar** (1 min) — Tap venue, view availability, confirm Aug 20
4. **Payment** (1 min) — PayBox test card → push notification → tap → deep-link into booking
5. **Vendor side** (1 min) — Log in as vendor, see new booking in analytics, revenue chart updates
6. **Admin moderation** (30s) — Web admin moderates flagged review
7. **Architecture trace** (1 min) — `docker compose logs` — saga `core → payment.Charge → core.MarkBookingPaid`
8. **Q & A** (1 min)

Roadmap

Tier 1 — ships before defense

- AI event planner with tool use
- Vendor analytics dashboard
- KZ / RU / EN i18n completion
- Push notifications with deep-links
- Chat polish (typing · read receipts)

Tier 2 — next iteration

- Smart vendor recommendations (embeddings)
- Dynamic pricing (peak / off-peak)
- Google Calendar sync
- Referral · promo codes
- Verified-booking review badge

Tier 3 — research — escrow with milestone release · voice input · ML matching · KYC via egov.kz · AI-generated minute-by-minute event runbook

Lessons Learned

What worked	What I would change
One DB per service — clear ownership	Synchronous payment saga is fragile; should publish events (NATS / Kafka)
Riverpod for Flutter — fewer rebuilds	Started with hand-rolled DTOs; should generate from OpenAPI from day 1
FSD on web — features stay isolated	Tailwind v4 + Next.js 16 still rough; some hydration warnings remain
Claude + MCP — 2–3× productivity	Need stricter prompt discipline; early sessions drifted scope
Per-service Go modules — independent CI	Workspace <code>replace</code> directives are subtle; broke CI twice
gRPC for internal — no API drift	Browser debugging painful; we keep REST at the gateway for a reason

Questions?

Repo · github.com/Bahaidahar/diploma

Architecture deep-dive · [presentation/research/code-map.md](#)

Comparative analysis · [presentation/research/competitors.md](#)

Stack justification · [presentation/research/stack-justification.md](#)